SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **M Nishanth**

Annexure A
Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — EXPERIMENTS

Experiment 1: MD5 vs SHA-256 — Hashing, Collision Resistance and Performance Comparison

Aim

To implement MD5 and SHA-256 hashing in Python, hash multiple input messages, and analyze the two algorithms' collision resistance and performance characteristics.

Understanding of Concepts

- MD5 produces a 128-bit digest and SHA-256 produces a 256-bit digest; a larger digest space directly increases resistance to brute-force collision search (birthday-bound attacks).
- The avalanche effect is a core requirement of cryptographic hash functions: a single-character change in the input should change roughly half of the output bits.
- MD5 has known practical collision vulnerabilities (Wang et al., 2004), while SHA-256 has no known practical collision attacks to date.

Tools / Libraries Used

Python 3 hashlib (built-in) for MD5 and SHA-256 computation; time module for benchmarking.

Python Implementation

```
"""
Experiment 1: Implement and compare MD5 and SHA-256 by hashing multiple
input messages. Analyze collision resistance and performance differences.
"""
import hashlib
import time

messages = [
    "Hello World",
    "Hello World!",          # single character difference (avalanche test)
    "Cryptography and Network Security",
    "The quick brown fox jumps over the lazy dog",
    "SIMATS Engineering CSA51",
]

def hash_message(message, algo):
    h = hashlib.new(algo)
    h.update(message.encode())
    return h.hexdigest()

print("=" * 90)
print(f"{'Message':40} | {'MD5 Digest':32} | Algo")
print("=" * 90)
for msg in messages:
    md5_digest = hash_message(msg, "md5")
    sha_digest = hash_message(msg, "sha256")
    print(f"{msg[:38]:40} | {md5_digest}")
    print(f"{'':40} | SHA-256: {sha_digest}")
    print("-" * 90)
```

```
# Avalanche effect demonstration: compare hash of two near-identical strings
print("\nAVALANCHE EFFECT CHECK (Hello World vs Hello World!)")
m1, m2 = messages[0], messages[1]
md5_1, md5_2 = hash_message(m1, "md5"), hash_message(m2, "md5")
sha_1, sha_2 = hash_message(m1, "sha256"), hash_message(m2, "sha256")
print(f"MD5 ({m1!r}):    {md5_1}")
print(f"MD5 ({m2!r}):   {md5_2}")
print(f"SHA256 ({m1!r}): {sha_1}")
print(f"SHA256 ({m2!r}):{sha_2}")

def bit_diff_percentage(hex1, hex2):
    b1 = bin(int(hex1, 16))[2:].zfill(len(hex1) * 4)
    b2 = bin(int(hex2, 16))[2:].zfill(len(hex2) * 4)
    diff = sum(c1 != c2 for c1, c2 in zip(b1, b2))
    return round(100 * diff / len(b1), 2)

print(f"\nBit difference MD5:    {bit_diff_percentage(md5_1, md5_2)}% of output bits changed")
print(f"Bit difference SHA-256: {bit_diff_percentage(sha_1, sha_2)}% of output bits changed")

# Performance comparison
print("\nPERFORMANCE COMPARISON (100,000 iterations on a fixed message)")
test_msg = b"Performance test message for hashing benchmark"
iterations = 100_000

start = time.perf_counter()
for _ in range(iterations):
    hashlib.md5(test_msg).hexdigest()
md5_time = time.perf_counter() - start

start = time.perf_counter()
for _ in range(iterations):
    hashlib.sha256(test_msg).hexdigest()
sha_time = time.perf_counter() - start

print(f"MD5:    {iterations} hashes in {md5_time:.4f} sec -> {iterations/md5_time:,.0f}
hashes/sec")
print(f"SHA256: {iterations} hashes in {sha_time:.4f} sec -> {iterations/sha_time:,.0f}
hashes/sec")
print(f"MD5 is approximately {sha_time/md5_time:.2f}x the speed of SHA-256" if md5_time < sha_time
      else f"SHA-256 is approximately {md5_time/sha_time:.2f}x the speed of MD5")
```

Output

```
================================================================================================
Message                                  | MD5 Digest                        | Algo
================================================================================================
Hello World                              | b10a8db164e0754105b7a99be72e3fe5
                                         | SHA-256:
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
------------------------------------------------------------------------------------------------
Hello World!                             | ed076287532e86365e841e92bfc50d8c
                                         | SHA-256:
7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
------------------------------------------------------------------------------------------------
Cryptography and Network Security        | 1ee455e1135655f44979ef4d15bc3563
                                         | SHA-256:
e7436c91d475cf9f1e0bfaa6533e62ae7bc5dd5d6911760cc157fd14f10d06fe
------------------------------------------------------------------------------------------------
The quick brown fox jumps over the laz   | 9e107d9d372bb6826bd81d3542a419d6
                                         | SHA-256:
d7a8fbb307d7809469ca9abcb0082e4f8d5651e46d3cdb762d02d0bf37c9e592
------------------------------------------------------------------------------------------------
SIMATS Engineering CSA51                 | 864babc9e15171ca7a13e9175cbbfc31
                                         | SHA-256:
8230a1beee0b58707bdef5209361a9da49da6e50d7e1da44cde5b026645036cd
------------------------------------------------------------------------------------------------

AVALANCHE EFFECT CHECK (Hello World vs Hello World!)
```

```
MD5 ('Hello World'):    b10a8db164e0754105b7a99be72e3fe5
MD5 ('Hello World!'):   ed076287532e86365e841e92bfc50d8c
SHA256 ('Hello World'): a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
SHA256 ('Hello World!'):7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069

Bit difference MD5:    57.03% of output bits changed
Bit difference SHA-256: 48.05% of output bits changed

PERFORMANCE COMPARISON (100,000 iterations on a fixed message)
MD5:    100000 hashes in 0.0457 sec -> 2,187,603 hashes/sec
SHA256: 100000 hashes in 0.0398 sec -> 2,513,305 hashes/sec
SHA-256 is approximately 1.15x the speed of MD5
```

Analysis of Results

- Both algorithms exhibited a strong avalanche effect: changing one character ('Hello World' -> 'Hello World!') altered roughly half the output bits in both MD5 (~57%) and SHA-256 (~48%), confirming both satisfy basic diffusion requirements.
- Despite passing the avalanche test, MD5 remains cryptographically broken at the collision-resistance level due to structural weaknesses in its compression function, which the avalanche test alone does not reveal.
- Performance testing over 100,000 iterations showed MD5 and SHA-256 have comparable raw throughput on this hardware, with SHA-256 in fact executing slightly faster in this run — demonstrating that SHA-256's stronger security does not come at a meaningful performance cost on modern hardware, removing 'MD5 is faster' as a valid justification for continued use.
- Conclusion: SHA-256 should be preferred over MD5 for any security-relevant hashing (integrity checks, password derivation building blocks, digital signatures) since it offers materially stronger collision resistance at essentially no performance penalty.

Experiment 2: RSA Digital Signature — Signing and Verification

Aim

To develop a Python program implementing an RSA-based digital signature scheme, demonstrate message signing and verification, and analyze its effectiveness for integrity and authentication.

Understanding of Concepts

- A digital signature is generated by hashing the message and encrypting the hash with the signer's private key; anyone can verify it using the signer's public key.
- RSA signatures rely on the computational hardness of factoring large integers (the RSA problem) for their security.
- PKCS#1 v1.5 padding (used here via perceptome’s pkcs1_15) structures the hash before the RSA operation to prevent trivial forgeries.

Tools / Libraries Used

PyCryptodome (Crypto.PublicKey.RSA, Crypto.Signature.pkcs1_15, Crypto.Hash.SHA256) for RSA-2048 key generation, signing, and verification.

Python Implementation

```
"""
Experiment 2: Digital Signature scheme using RSA.
Demonstrates message signing and signature verification.
"""
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

# Step 1: Key generation
key = RSA.generate(2048)
private_key = key
public_key = key.publickey()
print("RSA-2048 key pair generated.")
print(f"Public exponent (e): {public_key.e}")
print(f"Modulus bit length: {public_key.n.bit_length()} bits\n")

def sign_message(message: bytes, priv_key):
    h = SHA256.new(message)
    signature = pkcs1_15.new(priv_key).sign(h)
    return signature

def verify_signature(message: bytes, signature: bytes, pub_key):
    h = SHA256.new(message)
    try:
        pkcs1_15.new(pub_key).verify(h, signature)
        return True
    except (ValueError, TypeError):
        return False

message = b"Transfer INR 50,000 to Account 1234567890"
print(f"Original message: {message.decode()}")

signature = sign_message(message, private_key)
print(f"Signature (hex, first 64 chars): {signature.hex()[:64]}...")

# Legitimate verification
result_valid = verify_signature(message, signature, public_key)
print(f"\nVerification with correct message & correct public key: {'VALID' if result_valid else
'INVALID'}")

# Tampered message test
tampered_message = b"Transfer INR 90,000 to Account 1234567890"
result_tampered = verify_signature(tampered_message, signature, public_key)
print(f"Verification after message tampering (amount changed): {'VALID' if result_tampered else
'INVALID (tampering detected)'}")

# Wrong key test (simulate attacker's key)
attacker_key = RSA.generate(2048)
result_wrong_key = verify_signature(message, signature, attacker_key.publickey())
print(f"Verification using an unrelated public key: {'VALID' if result_wrong_key else 'INVALID
(forgery rejected)'}")

# Non-repudiation demonstration
print("\nNon-repudiation check: only the holder of the private key could have produced")
print("a signature that verifies under the corresponding public key -- the signer")
print("cannot deny having signed the original message.")
```

Output

```
RSA-2048 key pair generated.
Public exponent (e): 65537
Modulus bit length: 2048 bits

Original message: Transfer INR 50,000 to Account 1234567890
Signature (hex, first 64 chars):
5b9794efaecc85973f44747114336243acfcc16133a72094bc361e550aa4a7b9...

Verification with correct message & correct public key: VALID
Verification after message tampering (amount changed): INVALID (tampering detected)
Verification using an unrelated public key: INVALID (forgery rejected)

Non-repudiation check: only the holder of the private key could have produced
a signature that verifies under the corresponding public key -- the signer
cannot deny having signed the original message.
```

Analysis of Results

- The signature verified successfully only when both the original message and the correct public key were used, confirming the scheme correctly binds a signature to a specific message and signer.
- Tampering with even one digit of the transaction amount caused verification to fail, demonstrating that RSA signatures reliably detect integrity violations because the underlying SHA-256 hash changes completely for any modification.
- Verification with an unrelated (attacker's) public key failed as expected, showing the scheme correctly rejects forged signatures and provides authentication of the true signer's identity.
- Because only the private-key holder could have produced a signature verifiable under the corresponding public key, the scheme also provides non-repudiation — the signer cannot later deny authorizing the transaction.

Experiment 3: HMAC vs Simple Hash — Message Authentication Comparison

Aim

To simulate HMAC using SHA-256 and compare its security properties against a simple, unkeyed hash-based integrity check.

Understanding of Concepts

- A plain hash H(message) is publicly computable by anyone, so it can only detect accidental corruption — it cannot authenticate the sender because an attacker can simply recompute a new hash after modifying the message.
- HMAC introduces a shared secret key into a nested hash construction, HMAC(K, m) = H((K XOR opad) || H((K XOR ipad) || m)), which an attacker cannot reproduce without knowledge of the key.

- The nested structure of HMAC also defeats length-extension attacks that affect Merkle–Damgård hash functions such as SHA-256 when used naively (e.g., H(key || message)).

Tools / Libraries Used

Python 3 hmac and hashlib (built-in) modules using SHA-256 as the underlying hash.

Python Implementation

```
"""
Experiment 3: Simulate HMAC using SHA-256 and compare with a simple
(unkeyed) hash-based integrity check.
"""
import hashlib
import hmac as hmac_lib

SECRET_KEY = b"super-secret-shared-key-2026"

def simple_hash_integrity(message: bytes):
    """Naive integrity check: just hash the message (no secret)."""
    return hashlib.sha256(message).hexdigest()

def hmac_integrity(message: bytes, key: bytes):
    """Proper keyed HMAC integrity check."""
    return hmac_lib.new(key, message, hashlib.sha256).hexdigest()

message = b"Account balance: 100000"
print(f"Original message: {message.decode()}")

# --- Simple hash approach ---
simple_tag = simple_hash_integrity(message)
print(f"\n[Simple Hash] SHA-256(message) = {simple_tag}")

# Attacker intercepts, modifies message AND recomputes the hash (no key needed)
tampered_message = b"Account balance: 900000"
attacker_forged_tag = simple_hash_integrity(tampered_message)
print(f"[Attack] Attacker changes message to: {tampered_message.decode()}")
print(f"[Attack] Attacker recomputes hash themselves: {attacker_forged_tag}")
print("[Result] Receiver has no way to distinguish this from a legitimate hash --")
print("         the simple hash provides NO authentication, only accidental-error detection.")

# --- HMAC approach ---
print("\n" + "=" * 70)
hmac_tag = hmac_integrity(message, SECRET_KEY)
print(f"[HMAC-SHA256] HMAC(key, message) = {hmac_tag}")

# Attacker (without the key) tries to forge a tag for the tampered message
wrong_key_guess = b"attacker-guessed-key"
attacker_hmac_attempt = hmac_integrity(tampered_message, wrong_key_guess)
print(f"[Attack] Attacker tampers message to: {tampered_message.decode()}")
print(f"[Attack] Attacker cannot know SECRET_KEY, so their forged HMAC = {attacker_hmac_attempt}")

# Legitimate receiver verifies using the real key
receiver_check = hmac_integrity(tampered_message, SECRET_KEY)
is_valid = hmac_lib.compare_digest(receiver_check, attacker_hmac_attempt)
print(f"[Verify] Receiver recomputes HMAC with real key over tampered message: {receiver_check}")
print(f"[Verify] Does attacker's forged tag match? {'YES (compromised)' if is_valid else 'NO ->
tampering detected & rejected'}")

# Correct verification path
original_check = hmac_lib.compare_digest(hmac_integrity(message, SECRET_KEY), hmac_tag)
print(f"\n[Verify] Original untampered message verifies correctly: {original_check}")

print("\nCONCLUSION:")
print("- Simple hash: anyone can recompute a valid-looking hash after tampering -> no
authentication.")
```

```
print("- HMAC: requires the shared secret key to produce a valid tag -> provides both")
print("  integrity AND authenticity, and resists length-extension attacks via its nested
construction.")
```

Output

```
Original message: Account balance: 100000

[Simple Hash] SHA-256(message) = 94ec8609a696482a1b613762b4bc8b597fa8d064aa9f823724a92c87e4714074
[Attack] Attacker changes message to: Account balance: 900000
[Attack] Attacker recomputes hash themselves:
d6ceb2ae56eacbfaaa8f26503e2a14658586a5777821bf3e0983829dfa9d016e
[Result] Receiver has no way to distinguish this from a legitimate hash --
         the simple hash provides NO authentication, only accidental-error detection.

======================================================================
[HMAC-SHA256] HMAC(key, message) =
dc8f742e9e830b8295363d558bfc8c13c78a2d3f23256a39217e288e3a078782
[Attack] Attacker tampers message to: Account balance: 900000
[Attack] Attacker cannot know SECRET_KEY, so their forged HMAC =
2f1527eb2df0e03ca3c691bc8fa5b3a2203ae305c638386d844061911fa61e05
[Verify] Receiver recomputes HMAC with real key over tampered message:
d4dfd144dd61ac74cb6832e39c52b47fa44a92e380b0281f8773ab292caf8a12
[Verify] Does attacker's forged tag match? NO -> tampering detected & rejected

[Verify] Original untampered message verifies correctly: True

CONCLUSION:
- Simple hash: anyone can recompute a valid-looking hash after tampering -> no authentication.
- HMAC: requires the shared secret key to produce a valid tag -> provides both
  integrity AND authenticity, and resists length-extension attacks via its nested construction.
```

Analysis of Results

- For the simple hash scheme, the simulated attacker was able to modify the account balance and independently compute a new, self-consistent hash — the receiver has no cryptographic basis to reject the forged message.
- For the HMAC scheme, the attacker (without the secret key) could not produce a tag that matched the value the legitimate receiver computed with the real key over the tampered message; the mismatch caused the tampering to be correctly detected and rejected.
- The original, untampered message correctly verified under HMAC using the shared key, confirming the scheme does not produce false rejections for legitimate traffic.
- This demonstrates that integrity alone (simple hash) is insufficient in an adversarial setting; authenticity requires a keyed construction such as HMAC.

Experiment 4: Simplified Kerberos Simulation — Ticket Issuance and Replay-Attack Resistance

Aim

To implement a simplified Kerberos authentication mechanism simulating ticket generation, distribution, and validation between a client and server, and to analyze its effectiveness against replay attacks.

Understanding of Concepts

- Kerberos uses a trusted third party (the Key Distribution Center, split conceptually here into an Authentication Server and Ticket Granting Server) to issue time-limited tickets rather than having the client repeatedly prove its password to every service.
- A Ticket Granting Ticket (TGT) lets the client request Service Tickets without re-entering credentials; a Service Ticket then grants access to a specific application server.
- Replay protection in real Kerberos relies on authenticators containing a timestamp/nonce that must be both fresh (within a short validity window) and not previously seen by the verifying server.

Tools / Libraries Used

Python 3 standard library only: hashlib (to simulate encrypted/sealed tickets via keyed hashing), time, secrets, and json.

Python Implementation

```
"""
Experiment 4: Simplified Kerberos authentication mechanism simulating
ticket generation, distribution, and validation between a client and
server, including a replay-attack test.
"""
import hashlib
import time
import secrets
import json

class AuthenticationServer:
    """Issues Ticket Granting Tickets (TGT) after verifying client credentials."""
    def __init__(self):
        # In real Kerberos this would be a KDC database of client secret keys
        self.client_keys = {"alice": "alice_secret_key"}
        self.tgs_key = "tgs_secret_key"  # shared between AS and TGS

    def authenticate(self, client_id, password):
        if self.client_keys.get(client_id) != password:
            return None
        session_key = secrets.token_hex(16)
        tgt = {
            "client_id": client_id,
            "session_key": session_key,
            "issued_at": time.time(),
            "expires_at": time.time() + 300,  # 5 minute validity
        }
```

```
        # TGT is "encrypted" for the TGS (simulated using a keyed hash wrapper)
        tgt_sealed = self._seal(tgt, self.tgs_key)
        print(f"[AS] Client '{client_id}' authenticated. TGT issued (valid 300s).")
        return tgt_sealed, session_key

    def _seal(self, data: dict, key: str):
        payload = json.dumps(data, sort_keys=True)
        tag = hashlib.sha256((key + payload).encode()).hexdigest()
        return {"payload": payload, "tag": tag}


class TicketGrantingServer:
    """Validates the TGT and issues a Service Ticket for the target server."""
    def __init__(self, tgs_key):
        self.tgs_key = tgs_key
        self.service_key = "service_secret_key"
        self.seen_authenticators = set()  # replay cache

    def request_service_ticket(self, tgt_sealed, authenticator):
        # Verify TGT integrity
        expected_tag = hashlib.sha256((self.tgs_key + tgt_sealed["payload"]).encode()).hexdigest()
        if expected_tag != tgt_sealed["tag"]:
            print("[TGS] TGT integrity check FAILED -> request rejected.")
            return None
        tgt = json.loads(tgt_sealed["payload"])

        if time.time() > tgt["expires_at"]:
            print("[TGS] TGT expired -> request rejected.")
            return None

        # Replay protection: authenticator (nonce+timestamp) must be fresh AND unused
        auth_id = authenticator["nonce"]
        if auth_id in self.seen_authenticators:
            print("[TGS] REPLAY DETECTED: authenticator already used -> request rejected.")
            return None
        if time.time() - authenticator["timestamp"] > 5:  # 5 second freshness window
            print("[TGS] Authenticator expired (outside freshness window) -> rejected.")
            return None

        self.seen_authenticators.add(auth_id)
        service_session_key = secrets.token_hex(16)
        service_ticket = {
            "client_id": tgt["client_id"],
            "session_key": service_session_key,
            "issued_at": time.time(),
            "expires_at": time.time() + 120,
        }
        service_ticket_sealed = self._seal(service_ticket, self.service_key)
        print(f"[TGS] Authenticator verified & fresh. Service Ticket issued for
'{tgt['client_id']}'.")
        return service_ticket_sealed, service_session_key

    def _seal(self, data: dict, key: str):
        payload = json.dumps(data, sort_keys=True)
        tag = hashlib.sha256((key + payload).encode()).hexdigest()
        return {"payload": payload, "tag": tag}


class ApplicationServer:
    """Validates the service ticket presented by the client."""
    def __init__(self, service_key):
        self.service_key = service_key

    def validate(self, service_ticket_sealed):
        expected_tag = hashlib.sha256((self.service_key +
service_ticket_sealed["payload"]).encode()).hexdigest()
        if expected_tag != service_ticket_sealed["tag"]:
            print("[Server] Service ticket integrity FAILED -> access denied.")
            return False
```

```
        ticket = json.loads(service_ticket_sealed["payload"])
        if time.time() > ticket["expires_at"]:
            print("[Server] Service ticket expired -> access denied.")
            return False
        print(f"[Server] Access GRANTED to '{ticket['client_id']}'.")
        return True


def make_authenticator(client_id, session_key):
    """Client builds a fresh authenticator: a nonce + timestamp bound to the session key."""
    return {
        "client_id": client_id,
        "nonce": secrets.token_hex(8),
        "timestamp": time.time(),
    }


print("=" * 70)
print("PHASE 1: Normal authentication flow")
print("=" * 70)
AS = AuthenticationServer()
tgs_key = AS.tgs_key
TGS = TicketGrantingServer(tgs_key)
server_key = TGS.service_key
App = ApplicationServer(server_key)

tgt, tgt_session_key = AS.authenticate("alice", "alice_secret_key")
authenticator1 = make_authenticator("alice", tgt_session_key)
service_ticket, svc_session_key = TGS.request_service_ticket(tgt, authenticator1)
App.validate(service_ticket)

print("\n" + "=" * 70)
print("PHASE 2: Replay attack simulation (attacker resends captured authenticator)")
print("=" * 70)
print("[Attacker] Captured authenticator1 from the wire, resending identical request...")
replay_result = TGS.request_service_ticket(tgt, authenticator1)
if replay_result is None:
    print("[Result] Replay attack was BLOCKED by the TGS replay cache.")

print("\n" + "=" * 70)
print("PHASE 3: Legitimate client makes a new request with a fresh authenticator")
print("=" * 70)
authenticator2 = make_authenticator("alice", tgt_session_key)
service_ticket2, _ = TGS.request_service_ticket(tgt, authenticator2)
if service_ticket2:
    App.validate(service_ticket2)

print("\n" + "=" * 70)
print("PHASE 4: Stale authenticator simulation (outside freshness window)")
print("=" * 70)
stale_authenticator = make_authenticator("alice", tgt_session_key)
stale_authenticator["timestamp"] = time.time() - 30  # 30 seconds old
result_stale = TGS.request_service_ticket(tgt, stale_authenticator)
if result_stale is None:
    print("[Result] Stale authenticator correctly rejected (freshness window exceeded).")
```

Output

```
======================================================================
PHASE 1: Normal authentication flow
======================================================================
[AS] Client 'alice' authenticated. TGT issued (valid 300s).
[TGS] Authenticator verified & fresh. Service Ticket issued for 'alice'.
[Server] Access GRANTED to 'alice'.

======================================================================
PHASE 2: Replay attack simulation (attacker resends captured authenticator)
```

```
======================================================================
[Attacker] Captured authenticator1 from the wire, resending identical request...
[TGS] REPLAY DETECTED: authenticator already used -> request rejected.
[Result] Replay attack was BLOCKED by the TGS replay cache.

======================================================================
PHASE 3: Legitimate client makes a new request with a fresh authenticator
======================================================================
[TGS] Authenticator verified & fresh. Service Ticket issued for 'alice'.
[Server] Access GRANTED to 'alice'.

======================================================================
PHASE 4: Stale authenticator simulation (outside freshness window)
======================================================================
[TGS] Authenticator expired (outside freshness window) -> rejected.
[Result] Stale authenticator correctly rejected (freshness window exceeded).
```

Analysis of Results

- In the normal flow, the client successfully obtained a TGT, exchanged it for a Service Ticket using a fresh authenticator, and was granted access by the application server — confirming the three-party protocol functions correctly end to end.
- When the same authenticator was replayed by a simulated attacker, the Ticket Granting Server's replay cache correctly identified the duplicate nonce and rejected the request, directly demonstrating replay-attack prevention.
- A subsequent legitimate request using a newly generated authenticator succeeded normally, showing the replay cache does not block legitimate repeated use of the same TGT, only reuse of the same authenticator.
- An authenticator artificially aged outside the 5-second freshness window was also rejected, showing that even an unused (non-replayed) authenticator is unacceptable once it is stale, matching Kerberos's real-world reliance on tight clock synchronization.
- This confirms that replay resistance in Kerberos depends on two independent controls working together: a bounded freshness window and a server-side cache of already-seen authenticators — removing either one (as in the vulnerable scenario in the case study) reopens the replay vector.

Experiment 5: ElGamal Digital Signature — Implementation and Security Evaluation

Aim

To develop a Python implementation of the ElGamal digital signature scheme and evaluate its security properties compared to standard (RSA-based) digital signature approaches.

Understanding of Concepts

- ElGamal signatures derive their security from the difficulty of the Discrete Logarithm Problem (DLP) in a finite cyclic group, in contrast to RSA's reliance on integer factorization.
- A safe prime p = 2q + 1 (with q also prime) is used so that the group order's factorization is trivially known, which is required to efficiently verify that a chosen generator g has the full order p-1.
- Each signature requires a freshly generated, secret, unpredictable random value k; reusing k across two signatures is a catastrophic, well-documented weakness that allows direct recovery of the private key.

Tools / Libraries Used

Python 3 hashlib and random (built-in); sympy.isprime for primality testing during safe-prime generation.

Python Implementation

```
"""
Experiment 5: ElGamal digital signature scheme implementation and
evaluation against standard (RSA-style) digital signature approaches.
"""
import hashlib
import random
from sympy import isprime

def generate_safe_prime(bits=160):
    """Generate a safe prime p = 2q + 1 (q prime) so that p-1 = 2*q factors trivially,
    making primitive-root verification fast."""
    while True:
        q = random.getrandbits(bits - 1) | 1
        if isprime(q):
            p = 2 * q + 1
            if isprime(p):
                return p, q

def find_generator(p, q):
    """For a safe prime p = 2q+1, an element g is a generator of Z_p* iff
    g^2 != 1 mod p and g^q != 1 mod p."""
    while True:
        g = random.randrange(2, p - 1)
        if pow(g, 2, p) != 1 and pow(g, q, p) != 1:
            return g

def hash_message(message: bytes, mod):
    digest = hashlib.sha256(message).digest()
    return int.from_bytes(digest, "big") % mod

def modinv(a, m):
```

```
    return pow(a, -1, m)

def key_generation(bits=160):
    p, q = generate_safe_prime(bits)
    g = find_generator(p, q)
    x = random.randrange(2, p - 2)       # private key
    y = pow(g, x, p)                     # public key component
    return {"p": p, "g": g, "y": y}, x

def sign(message: bytes, params, x):
    p, g = params["p"], params["g"]
    while True:
        k = random.randrange(2, p - 2)
        if __import__("math").gcd(k, p - 1) == 1:
            break
    h = hash_message(message, p)
    r = pow(g, k, p)
    k_inv = modinv(k, p - 1)
    s = (k_inv * (h - x * r)) % (p - 1)
    return (r, s)

def verify(message: bytes, signature, params):
    p, g, y = params["p"], params["g"], params["y"]
    r, s = signature
    if not (0 < r < p):
        return False
    h = hash_message(message, p)
    left = (pow(y, r, p) * pow(r, s, p)) % p
    right = pow(g, h, p)
    return left == right

print("Generating ElGamal domain parameters and key pair (160-bit safe prime)...")
public_params, private_key = key_generation(bits=160)
print(f"Prime p (bit length): {public_params['p'].bit_length()} bits")
print(f"Generator g: {public_params['g']}")
print(f"Public key y = g^x mod p: {str(public_params['y'])[:40]}...")
print(f"Private key x: (kept secret)\n")

message = b"Authenticated document approval - Version 1"
print(f"Message: {message.decode()}")

signature = sign(message, public_params, private_key)
print(f"Signature (r, s):\n  r = {str(signature[0])[:40]}...\n  s = {str(signature[1])[:40]}...")

is_valid = verify(message, signature, public_params)
print(f"\nVerification with original message: {'VALID' if is_valid else 'INVALID'}")

tampered = b"Authenticated document approval - Version 2"
is_valid_tampered = verify(tampered, signature, public_params)
print(f"Verification with tampered message:  {'VALID' if is_valid_tampered else 'INVALID
(tampering detected)'}")

# Demonstrate catastrophic failure if nonce k is reused (well-known ElGamal weakness)
print("\n--- Security property check: nonce (k) reuse vulnerability ---")
print("If the same random k is ever reused for two different messages, an attacker")
print("who observes both (message, signature) pairs can solve two linear equations")
print("in (x, k) and recover the private key x directly. This makes strict k-uniqueness")
print("and unpredictability essential -- unlike RSA-PSS, which does not depend on a")
print("fresh per-signature secret nonce for its core security.")

print("\n--- Comparison with standard (RSA) digital signatures ---")
print("- Key size: ElGamal typically needs larger parameters than RSA for equivalent security.")
print("- Signature size: ElGamal signatures (r, s) are roughly 2x the modulus size, larger than
RSA's single value.")
print("- Determinism: RSA-PKCS1v15 signing is deterministic; ElGamal signing is probabilistic
(depends on k),")
print("  so the same message signed twice yields two different, both-valid signatures.")
print("- Randomness dependency: ElGamal security critically depends on a fresh, secret,
unpredictable k")
print("  for every signature; RSA has no equivalent per-signature secret requirement.")
```

```
print("- Schnorr signatures (a related scheme) improve on ElGamal with shorter signatures and")
print("  provable security under the discrete-log assumption in the random oracle model.")
```

Output

```
Generating ElGamal domain parameters and key pair (160-bit safe prime)...
Prime p (bit length): 158 bits
Generator g: 187162185037927564535671495675086502993558857618
Public key y = g^x mod p: 153518672610651663756594530980987812796 9...
Private key x: (kept secret)

Message: Authenticated document approval - Version 1
Signature (r, s):
  r = 146071321192441702265281587427284872945 8...
  s = 246045560479576229736200797472779784580 2...

Verification with original message: VALID
Verification with tampered message:  INVALID (tampering detected)

--- Security property check: nonce (k) reuse vulnerability ---
If the same random k is ever reused for two different messages, an attacker
who observes both (message, signature) pairs can solve two linear equations
in (x, k) and recover the private key x directly. This makes strict k-uniqueness
and unpredictability essential -- unlike RSA-PSS, which does not depend on a
fresh per-signature secret nonce for its core security.

--- Comparison with standard (RSA) digital signatures ---
- Key size: ElGamal typically needs larger parameters than RSA for equivalent security.
- Signature size: ElGamal signatures (r, s) are roughly 2x the modulus size, larger than RSA's
single value.
- Determinism: RSA-PKCS1v15 signing is deterministic; ElGamal signing is probabilistic (depends on
k),
  so the same message signed twice yields two different, both-valid signatures.
- Randomness dependency: ElGamal security critically depends on a fresh, secret, unpredictable k
  for every signature; RSA has no equivalent per-signature secret requirement.
- Schnorr signatures (a related scheme) improve on ElGamal with shorter signatures and
  provable security under the discrete-log assumption in the random oracle model.
```

Analysis of Results

- The implementation correctly generated a safe-prime domain, a matching generator, and a key pair, then produced a signature (r, s) that verified successfully against the original message.
- Verification against a tampered message ('Version 1' changed to 'Version 2') correctly failed, confirming the scheme detects integrity violations in the same way RSA signatures do.
- Unlike RSA-PKCS1v15, which is deterministic, ElGamal signing is probabilistic because it depends on the random nonce k — signing the same message twice would produce two different, equally valid signatures.
- The experiment highlights a security trade-off: ElGamal's reliance on a fresh secret nonce per signature is an additional operational risk not present in RSA, making correct random-number generation critical to safe deployment; related schemes such as Schnorr signatures address this with a simpler, provably secure structure and shorter signatures for equivalent security levels.

Overall Conclusion

These five experiments collectively demonstrate the practical behavior of the cryptographic primitives underlying modern authentication and integrity systems. Hash function comparison (Experiment 1) shows why algorithm choice matters even when both functions pass basic diffusion tests. RSA and ElGamal signatures (Experiments 2 and 5) show two different mathematical foundations — factoring versus discrete logarithm — for achieving the same authentication and non-repudiation goals, each with distinct operational risks. The HMAC comparison (Experiment 3) isolates exactly why a bare hash cannot authenticate a message, motivating keyed constructions. Finally, the Kerberos simulation (Experiment 4) shows concretely how freshness windows and replay caches, working together, close the exact replay vulnerability described in the CO3 case study. Together, these results reinforce the course outcome of examining hash algorithms, digital signature schemes, and authentication frameworks for integrity, authentication, and non-repudiation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation